

Propuesta de Proyecto Integrador

1. Identificación

Nombre del proyecto	Go Gym
Curso	IIP323W — Tecnologías y Aplicaciones Web y Móviles
Trimestre	2026-3 (tercer trimestre)
Sección	1
Profesor	Cristóbal Maturana Ahumada
Fecha de entrega	Semana del 19 al 24 de octubre de 2026

2. Integrantes del grupo

Nombre completo	Correo UDD	Rol en el grupo
Vicente José Rodríguez Rojas	v.rodriquezr@udd.cl	Product Owner
Ian Harry Spikin Thomas	i.spikint@udd.cl	Backend Developer
Baptiste Vial Umaña	b.vialu@udd.cl	Frontend Developer

3. Problema a resolver

¿Qué problema están abordando?

Las personas que empiezan a ir al gimnasio suelen hacerlo con objetivos claros: bajar de peso, ganar músculo o simplemente adoptar hábitos más saludables. Si bien existen varias aplicaciones para esto, muchas se centran en uno solo de estos objetivos, cobran suscripción y tienen un diseño saturado y poco accesible.

En la Unidad 1 abordamos este problema con Go Gym en Ionic. En esta unidad reutilizamos esa misma app (mismo problema, usuarios, diseño y pantallas) y la llevamos a Flutter, sumando cuentas de usuario, entrenamientos guardados en la nube y un catálogo de ejercicios obtenido desde una API de deportes.

4. Usuario objetivo

¿Quién es el usuario? Describan al menos una persona (edad, contexto, necesidad).

Uno de los usuarios de la aplicación es un joven de entre 18 y 30 años que quiere comenzar a ir al gimnasio, pero no sabe cómo entrenar ni cómo realizar correctamente los ejercicios. Por esto, necesita una aplicación que lo guíe y le enseñe de forma sencilla.

Otro usuario es un adulto de entre 30 y 50 años que busca mantenerse activo o retomar el ejercicio, pero cuenta con poco tiempo. Necesita una aplicación que le entregue rutinas adaptadas a sus objetivos, disponibilidad y nivel de experiencia.

5. Propuesta de valor

Resuman en una frase: ¿por qué alguien usaría esta app en lugar de no usarla?

Es una alternativa barata, sencilla y portable para medir el progreso y motivarse a cumplir las metas personales.

6. Funcionalidades principales

Listen entre 3 y 5 funcionalidades core que debe tener el MVP.

- **Cuenta de usuario:** registrarse, iniciar sesión y cerrar sesión con Supabase; cada persona ve solo sus propios datos.
- **Biblioteca de ejercicios (API):** catálogo obtenido desde la API de wger, con nombre, grupo muscular e imagen; al tocar un ejercicio se abre su detalle con instrucciones, músculos y equipamiento.
- **Registrar un entrenamiento:** formulario con validación (ejercicio, peso, repeticiones, series) para crear, editar y borrar las sesiones guardadas.
- **Dashboard:** resumen de las últimas sesiones, racha y totales, que se actualiza al registrar un entrenamiento.
- **Perfil y calorías:** detalle del usuario, sus estadísticas y medidor de ingesta y quema de calorías.

Cómo se cubre lo que todos deben tener en la Unidad 2:

Requisito	Dónde se cumple en Go Gym
1. Login con Supabase	Pantallas de registro e inicio de sesión con correo y contraseña; «Cerrar sesión» en Perfil.
2. Datos de la API	Biblioteca de ejercicios: llamada GET a wger (exerciseinfo) mostrada con FutureBuilder, con sus estados de carga, error y datos.
3. Navegación	Tocar un ejercicio abre la pantalla de detalle, que recibe el objeto Ejercicio por su constructor.
4. Animaciones	Hero en la imagen del ejercicio (lista → detalle) y AnimatedContainer en los filtros por grupo muscular al seleccionarlos.
5. Tabla en Supabase	Tabla «entrenamientos» (ejercicio, peso, repeticiones, series, fecha) con RLS: leer, crear, editar y borrar exigen auth.uid() = user_id.
6. Formulario con validación	Form y TextFormField con validator en «Registrar entrenamiento» (campos obligatorios; números mayores que cero). El mismo formulario sirve para editar.
7. Estado con Provider	EntrenamientosProvider (ChangeNotifier) compartido entre Inicio, Registrar entrenamiento y Perfil.
8. Widgets propios	SesionCard, EjercicioCard y StatCard, cada uno en su archivo dentro de lib/widgets/.
9. GitHub	Repositorio compartido, con una rama por funcionalidad y commits de los tres integrantes.

7. Stack y dependencias

Tecnologías que usarán además de Flutter + Dart (APIs, librerías, BD local).

Núcleo

- **Supabase (paquete `supabase_flutter`):** autenticación con correo y contraseña, y base de datos Postgres con Row Level Security (RLS) para la tabla de entrenamientos de cada usuario.
- **API de wger (REST, open source):** es la API de deportes de la app: catálogo de ejercicios con grupo muscular, equipamiento, instrucciones e imágenes. Sus endpoints públicos no requieren API key; la consumimos con el paquete `http`.
- **provider:** manejo de estado con ChangeNotifier compartido entre pantallas.
- **Animaciones propias de Flutter (Hero, AnimatedContainer):** no requieren dependencias adicionales.
- **Git y GitHub:** control de versiones del proyecto.

Opcionales según alcance

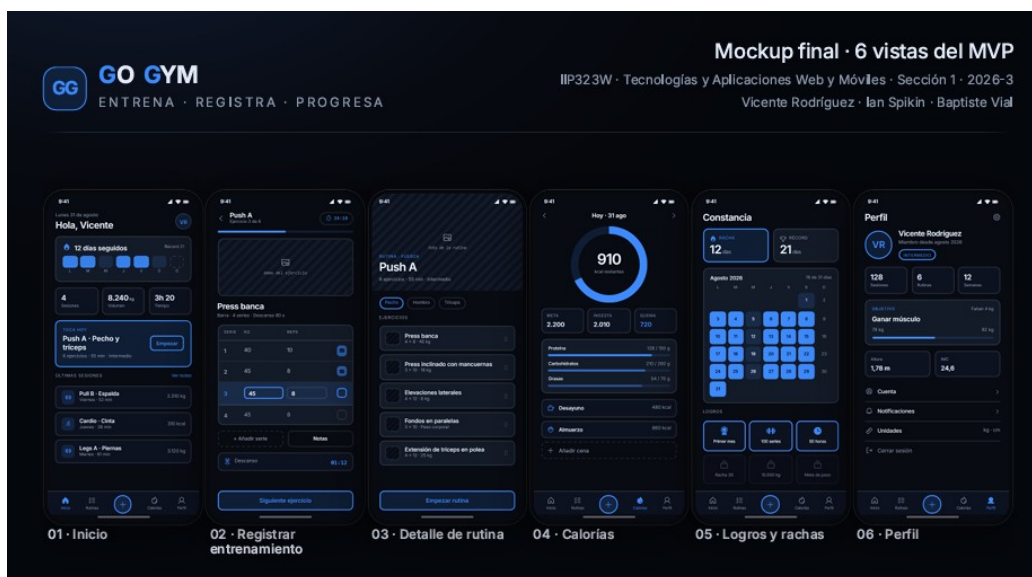
- **ExerciseDB (vía RapidAPI):** complemento a wger que trae GIFs demostrativos de cada ejercicio.
- **shared_preferences:** caché local del catálogo de ejercicios, para no depender de la red en cada apertura.
- **Health Connect (Android) y HealthKit (iOS):** lectura de frecuencia cardíaca y actividad mediante un plugin de Flutter, para afinar la estimación de calorías quemadas.
- **Nutritionix o Edamam:** base de datos de alimentos con calorías y macros, solo si se alcanza a cerrar el ciclo de calorías consumidas versus quemadas.

8. Diseño y mockup

Adjunten el link del mockup en Miro o Figma (deben ser visibles para el profesor con el link).

URL del mockup:

Se adjunta como imagen. Reutilizamos el mockup de la Unidad 1 (seis vistas del MVP); para Flutter se suman dos vistas: inicio de sesión / registro y biblioteca de ejercicios con su detalle.



9. Cronograma

Semana	Hito / Entregable	Responsable
1 (05-10)	Propuesta + Mockup inicial Repositorio en GitHub y proyecto en Supabase creados.	Vicente Rodríguez
2 (12-10)	Interfaz de la app (widgets Flutter) Pantallas llevadas desde Ionic, widgets propios, navegación con Hero y animación implícita. Login con Supabase.	Baptiste Vial (interfaz) Ian Spikin (login)
3 (19-10)	Consumo de API REST Biblioteca de ejercicios con wger y FutureBuilder. Tabla con RLS, formulario validado (CRUD) y Provider.	Ian Spikin (API y Supabase) Vicente Rodríguez (formulario y Provider)
4 (semana 19 al 24 oct)	Presentación + Control 2 App integrada y probada con dos cuentas distintas.	Todos

10. Riesgos y supuestos

¿Qué puede salir mal? ¿Qué están asumiendo?

Técnicos

- **Paso de Ionic a Flutter.** No hay conversión automática: la interfaz se reescribe en Dart, y eso puede consumir el tiempo destinado a los requisitos nuevos. **Mitigación:** reutilizar el mockup, la paleta y el modelo de datos ya definidos, y llevar primero el flujo principal (Inicio, Registrar entrenamiento, Perfil).
- **Dependencia de la API de wger.** Si el servicio falla o limita las peticiones, se cae la biblioteca de ejercicios. Además, no todos los ejercicios tienen imagen o traducción al español. **Mitigación:** manejar los estados de carga y error en el FutureBuilder, guardar el Future para no repetir la llamada en cada reconstrucción, usar una imagen de respaldo y mostrar el texto en inglés cuando falte la traducción.
- **RLS mal configurado.** Una política incorrecta puede exponer entrenamientos de otros usuarios o hacer que el guardado falle sin un mensaje claro. **Mitigación:** probar el CRUD con dos cuentas distintas antes de la presentación.
- **Plan gratuito de Supabase.** Los proyectos gratuitos se pausan tras una semana de baja actividad. **Mitigación:** revisar que el proyecto esté activo el día previo a la presentación.
- **Estado desincronizado.** El dashboard podría no reflejar un entrenamiento recién guardado. **Mitigación:** toda escritura pasa por el Provider, que avisa a las pantallas con notifyListeners().

De producto

- **Precisión del cálculo calórico.** Las estimaciones por MET tienen un margen de error considerable, sobre todo en entrenamiento de fuerza. **Mitigación:** presentar el valor como estimación, no como dato exacto.

- **Registro inconsistente.** La app depende de que el usuario anote cada serie y repetición.
Mitigación: minimizar la fricción del formulario, con valores precargados de la sesión anterior.

Legales y de privacidad

- **Datos sensibles.** Los datos de entrenamiento y salud son sensibles, y la entrada en vigencia de la Ley 21.719 de protección de datos personales está prevista para el 1 de diciembre de 2026.
Mitigación: pedir solo los datos necesarios, aislar los de cada usuario con RLS y permitir que borre sus registros.

De proyecto

- **Alcance amplio para el tiempo disponible.** Son nueve requisitos en pocas semanas. **Mitigación:** priorizar los nueve requisitos de la unidad y dejar las integraciones opcionales (salud, nutrición) para una segunda iteración.
- **Trabajo concentrado en una persona.** La unidad exige commits de todos los integrantes.
Mitigación: tareas asignadas por integrante desde la semana 1 y una rama por funcionalidad.

Supuestos

- Hay conexión a internet para iniciar sesión, consultar la API y guardar los entrenamientos; el modo offline queda para una iteración posterior.
- Los planes gratuitos de Supabase y de la API de wger alcanzan para el volumen de desarrollo, pruebas y presentación.
- El usuario registra sus entrenamientos con regularidad y con datos razonablemente precisos.
- El público objetivo son personas adultas sanas, sin condiciones médicas que requieran supervisión clínica.
- Un usuario equivale a una cuenta; no se contempla el rol de entrenador que administre a varios alumnos.

11. Firmas / Confirmación

Al entregar este documento, el grupo confirma que los integrantes listados están comprometidos con el proyecto y conocen las fechas clave de la asignatura.

Firma del responsable del grupo: _____

Fecha: 05/10/2026